React Native Frontend Architecture & Complete UI Blueprint
AI Remote Control System

This document provides a complete frontend architecture plan for building a modern, attractive, scalable, real-time React Native mobile application for the AI-powered remote control system.

1. Frontend Vision

The mobile app acts as:
• Remote desktop controller
• AI assistant dashboard
• Real-time system monitor
• Code execution console
• File manager
• Live streaming interface
• Notification center

The app should feel like:
• A futuristic operating system
• Cyberpunk-inspired developer dashboard
• AI command center
• Minimal but powerful

---

2. Recommended Frontend Tech Stack

Core Framework:
• React Native
• Expo SDK
• TypeScript

Navigation:
• React Navigation v7
• Bottom Tabs
• Native Stack

State Management:
• Zustand (recommended)
OR
• Redux Toolkit

Real-Time Communication:
• Socket.IO Client

• WebSockets

Networking:
• Axios
• React Query / TanStack Query

Animations:
• React Native Reanimated
• Moti
• Lottie Animations

UI Libraries:
• NativeWind (Tailwind)
• React Native Paper
• Tamagui (optional)

Charts:
• Victory Native
• React Native SVG Charts

Streaming:
• react-native-webrtc

Forms:
• React Hook Form
• Zod validation

Authentication:
• JWT
• SecureStore
• Biometrics

Notifications:
• Expo Notifications

---

3. Complete Frontend Folder Structure

mobile-app/
■■■ src/
■ ■■■ screens/
■ ■ ■■■ auth/
■ ■ ■■■ dashboard/
■ ■ ■■■ terminal/
■ ■ ■■■ stream/
■ ■ ■■■ files/
■ ■ ■■■ ai/
■ ■ ■■■ settings/
■ ■ ■■■ monitoring/
■ ■
■ ■■■ components/
■ ■ ■■■ cards/
■ ■ ■■■ buttons/
■ ■ ■■■ modals/

■ ■ ■■■ charts/
■ ■ ■■■ stream/
■ ■ ■■■ terminal/
■ ■ ■■■ ai/
■ ■
■ ■■■ navigation/
■ ■■■ services/
■ ■■■ sockets/
■ ■■■ store/
■ ■■■ hooks/
■ ■■■ utils/
■ ■■■ constants/
■ ■■■ themes/
■ ■■■ types/
■ ■■■ ai/
■ ■■■ assets/

---

4. UI Design Philosophy

Design Style:
• Dark futuristic UI
• Glassmorphism
• Neon accents
• Smooth animations
• Minimal clutter
• Dashboard-first approach

Primary Colors:
• Dark background
• Cyan
• Purple
• Electric blue

Typography:
• Inter
• Space Grotesk
• JetBrains Mono

Animations:
• Page transitions
• Interactive command animations
• Live streaming effects
• Terminal typing effects

---

5. Core Screens Architecture

Authentication:
• Splash screen

• Login
• Register
• Device pairing

Main Dashboard:
• Connected devices
• CPU/GPU usage
• Running apps
• AI assistant quick actions
• Recent commands

Remote Desktop Screen:
• Live PC stream
• Touch mouse controls
• Keyboard overlay
• Fullscreen support

Terminal Screen:
• Command execution
• Real-time logs
• Syntax highlighting
• AI code analysis

AI Assistant Screen:
• ChatGPT-like interface
• Voice commands
• Suggested actions
• Autonomous workflows

File Manager:
• Browse PC files
• Upload/download
• Drag-and-drop interactions

Monitoring Screen:
• System metrics
• RAM usage
• Temperature
• Internet speed

---

6. Navigation Architecture

Root Navigation:
• Authentication Stack
• Main App Stack

Main App Tabs:
1. Dashboard
2. Remote
3. Terminal
4. AI Assistant
5. Settings

Nested Navigators:
• Device details
• File manager
• Monitoring pages
• Stream controls

---

7. State Management Architecture

Recommended: Zustand

Stores:
authStore
• user
• token
• permissions

deviceStore
• connected devices
• active device
• device status

streamStore
• video stream
• stream quality
• fullscreen state

terminalStore
• logs
• command history
• active sessions

aiStore
• chat history
• AI status
• workflows

notificationStore
• alerts
• system messages

---

8. WebSocket Frontend Architecture

Socket Responsibilities:
• Real-time terminal logs
• Screen streaming
• AI responses
• Device heartbeat
• Mouse/keyboard sync

Socket Events:
connect_device
disconnect_device
terminal_output
screen_frame
ai_response
system_alert
file_upload_progress
command_status

Architecture:
Socket Provider
→ Event Handlers
→ Zustand Store Updates
→ UI Rendering

---

9. Dashboard UI Blueprint

Dashboard Components:
• Device cards
• Live stats widgets
• AI quick command panel
• Recent activity feed
• System health cards
• Quick actions grid

Recommended Widgets:
• CPU graph
• RAM graph
• GPU monitor
• Network speed
• Battery status
• Active process list

Animations:
• Live pulsing indicators
• Animated stat counters
• Graph transitions

---

10. Remote Desktop UI

Features:
• Live desktop streaming
• Touchpad mode
• Mouse mode
• Keyboard shortcuts
• Gesture support

Controls:
• Tap = click
• Two-finger drag
• Pinch zoom
• Long press = right click

Overlay Components:
• Floating keyboard
• FPS indicator
• Network quality
• AI assistant overlay

---

11. Terminal UI Architecture

Terminal Features:
• Multiple tabs
• Colored logs
• Command suggestions
• AI debugging assistant
• Syntax highlighting

Components:
• TerminalOutput
• CommandInput
• AIExplainButton
• ErrorCard
• SessionTabs

Advanced Features:
• Saved commands
• Script runner
• Interactive shell

---

12. AI Assistant Frontend

Chat Interface:
• Streaming AI responses
• Tool execution previews
• Workflow builder
• Task timeline

Features:
• Voice commands
• Image understanding
• Code explanation
• AI-generated automation

Suggested AI Components:
• ChatBubble
• WorkflowCard
• AgentStatus
• AIReasoningPanel

---

13. File Manager Frontend

Capabilities:
• PC file browsing
• Upload/download
• Search
• Preview files
• Folder tree navigation

UI Components:
• FileCard
• FolderTree
• UploadProgressBar
• SearchBar
• ContextMenu

Advanced Features:
• AI file summarization
• Image previews
• Video streaming

---

14. Frontend API Service Layer

API Structure:

services/
■■■ auth.service.ts
■■■ device.service.ts
■■■ terminal.service.ts
■■■ ai.service.ts
■■■ stream.service.ts
■■■ file.service.ts

Responsibilities:
• API requests
• Error handling
• Token refresh
• Retry logic
• Caching

---

15. Realtime Streaming UI

Streaming Stack:
• react-native-webrtc
• WebRTC signaling
• Adaptive quality

Frontend Features:
• FPS display
• Bitrate monitor
• Fullscreen
• Picture-in-picture

Optimization:
• Frame buffering
• Hardware acceleration
• Compression handling

---

16. Notification System

Types:
• AI task completed
• Device offline
• Error alerts
• Stream disconnected
• Terminal finished

Notification Channels:
• Push notifications
• In-app banners
• Toast messages
• Real-time alerts

---

17. Frontend Security

Security Features:
• JWT authentication
• Secure token storage
• Biometric login
• Encrypted WebSockets
• Device authorization

Libraries:
• expo-secure-store
• expo-local-authentication

---

18. Frontend Performance Optimization

Optimization Techniques:
• Lazy loading
• Memoization
• Virtualized lists
• WebSocket batching
• Debouncing
• Image caching

Streaming Optimization:
• Adaptive bitrate
• Frame skipping
• Dynamic resolution

---

19. Suggested Frontend Development Phases

Phase 1:
• Auth screens
• Navigation
• Dashboard

Phase 2:
• WebSockets
• Device management
• Terminal UI

Phase 3:
• Screen streaming
• Mouse controls

Phase 4:
• AI assistant
• Automation workflows

Phase 5:
• Advanced animations
• Performance tuning
• Production deployment

---

20. Final Frontend Recommendations

Recommended Development Order:
1. UI design system
2. Authentication
3. Dashboard

4. WebSocket architecture
5. Device communication
6. Terminal execution
7. Streaming interface
8. AI integration

Key Goal:
Create an experience that feels:
• futuristic
• responsive
• powerful
• developer-focused
• AI-native

This frontend should resemble:
• A cyberpunk developer OS
• AI command center
• Next-generation remote desktop platform
